

Gesture-Controlled Robot Using Computer Vision

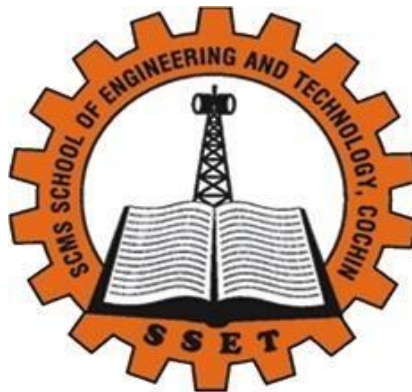
A SEMINAR REPORT

Submitted by

JOEL JOJU [SCM23MCA-2020]

to

The APJ Abdul Kalam Technological University
in partial fulfillment of the requirements for the award of the Degree of
Master of Computer Application



Department of Computer Science and Engineering
SCMS SCHOOL OF ENGINEERING AND TECHNOLOGY
(Affiliated to APJ Abdul Kalam Technological University)
VIDYA NAGAR, PALISSERY, KARUKUTTY
ERNAKULAM - 683582

APRIL 2025

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCMS SCHOOL OF ENGINEERING AND TECHNOLOGY

(Affiliated to APJ Abdul Kalam Technological University)

VIDYA NAGAR, PALISSERY, KARUKUTTY

ERNAKULAM - 683582



CERTIFICATE

This is to certify that the report entitled '**Gesture-Controlled Robot Using Computer Vision**' submitted by **JOEL JOJU** to the APJ Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the Degree of Master of Computer Application is a bonafide record of the seminar work carried out by him under my guidance and supervision.

Seminar Guide
Ms. Doney Daniel

Seminar Coordinator
Dr. Deepa K

Head Of The Department
Dr. Manish T I

ACKNOWLEDGEMENT

I am greatly indebted to **Dr. Anitha G Pillai**, Principal, SSET, Ernakulam and **Dr. Manish T I**, Head of department, Department of Computer Science and Engineering, SSET, who whole heartedly granted me the permission to do the seminar.

I would like to thank my guide, **Ms. Doney Daniel**, Assistant Professor, Department of Computer Science and Engineering, and seminar coordinator, **Dr. Deepa K**, Associate Professor, Department of Computer Science and Engineering, SSET who has given me valuable guidance and support all the way.

I would like to express my sincere gratitude to all the teachers of Computer Science Department who gave me moral and technical support. I would like to thank the supporting staff in the Computer lab whose dedicated work kept the lab working smoothly, thus enabling me to have access to various resources which helped me understand more about the seminar topic. I would also like to thank friends and family members for providing me with necessary resources and support. Last but not the least, I would like to thank God Almighty for helping me to do my seminar hasslefree.

ABSTRACT

In recent years, human-robot interaction (HRI) has improved a lot. This change has come from the need for robots to communicate more naturally with people. In the past, using touchscreens, keyboards, and joysticks to control robots was common. However, these methods were often confusing and didn't make things easy.

As technology advanced, speech recognition became a popular way to interact with robots. While it was a step forward, it had some problems. Robots sometimes struggled to understand different accents, background noises, and various languages.

Now, gesture recognition is getting more attention because it mimics how humans communicate with each other. With the help of computer vision and machine learning, robots can better understand and respond to human gestures. This improvement means that interactions with robots can feel more natural and smooth.

This seminar focuses on two main ways to use gestures to control robots. The first study introduces a deep-learning system called EUREKA. It utilizes a tool called Mediapipe to find hand landmarks and to predict gestures accurately. Remarkably, it can do this in real-time, even in different environments.

The second study talks about a real-time control system for robots. It also uses the MediaPipe algorithm, which detects 21 hand landmarks. This system lets users control a robot tracked with Raspberry Pi and Arduino technology.

Both studies show that robots can perform well in a variety of situations. Overall, these advancements highlight how computer vision can make gesture-controlled robots more accurate and effective in everyday use.

TABLE OF CONTENTS

CONTENTS	Page no.
Acknowledgement	i
Abstract	ii
List of figures	iv
List of tables	v
Chapter-1. Introduction	1
Chapter-2. Literature survey	2
Chapter-3. Proposed system	4
3.1 Efficient hand gesture recognition for human-robot interaction	4
3.1.1 Dataset	5
3.1.2 Working of eureka	5
3.2 Hand gestures robotic control based on computer vision	9
3.2.1 Hand gesture detection and classification:	9
3.2.2 Robot control	11
3.2.3 Hardware and software components:	11
Chapter-4. Methodology	13
4.1 MediaPipe	13
4.2 OpenCV	14
4.3 Python	15
4.4 Arduino	17
Chapter-5. Experiments & results	18
5.1 Efficient hand gesture recognition for human-robot interaction	18
5.2 Hand gestures robotic control based on computer vision	20
Chapter-6. Comparison results	24
Chapter-7. Conclusion	25
Chapter-8. References	26

LIST OF FIGURES

NO	TITLE	PAGE NO
3.1	Eureka architecture	5
3.2	Hand-gestures performed in front of ivo	8
3.3	Gestures	10
3.4	Base station	12
3.5	The assembled tank robot	12
4.1	20 hand landmarks	14
5.1	Batches with 30, 60 and 90 frames are taken	19

LIST OF TABLES

NO	TITLE	PAGE NO
5.1	Accuracy with ipn hand dataset	19
5.2	Light intensity variation based accuracy	21
5.3	Variations in distance-based accuracy	21
5.4	Tilt-angle testing accuracy	22
5.5	The robotic response time test	23
6.1	Comparison table	24

CHAPTER-1

INTRODUCTION

Human-robot interaction, or HRI [1], is an important area of study. As robots become more common in our daily lives—in places like healthcare, factories, and personal assistance—it's crucial to improve how we communicate with them. The aim of HRI is to make these conversations feel more natural and intuitive, helping robots understand what we mean and respond accordingly.

In the past, people interacted with robots mainly through physical tools like keyboards, joysticks, and touchscreens. However, these methods were not very flexible and often felt awkward. Then, speech recognition technology came along, allowing us to talk to robots. But this too had its problems, like misunderstandings caused by background noise, different languages, or various accents, making it less reliable in everyday situations.

Recently, gesture recognition has gained a lot of attention. This technology understands hand movements, similar to how we naturally communicate with each other. Using gestures to control robots can feel more intuitive and friendly. Thanks to improvements in computer vision and machine learning, gesture recognition has become faster and more accurate, allowing robots to respond in real-time [2].

In this report, we will dive deeper into how gesture-based control works for robots. By using computer vision and machine learning techniques, we can improve communication with robots. Our goal is to make it easier and more effective for people to use their hands to interact with robots.

CHAPTER-2

LITERATURE SURVEY

Qi et al. (2024) provide a comprehensive review of computer vision–based hand gesture recognition methods for human–robot interaction. Their survey discusses the evolution from classical image processing techniques to modern deep learning architectures, emphasizing the challenges posed by varying lighting conditions, background complexities, and real-time processing demands. The paper outlines emerging trends, including the integration of 3D hand pose estimation and temporal modeling, which are critical for improving recognition accuracy and robustness in dynamic environments.

Wameed and Alkamachi (2023) focus on a computer vision–based approach for robotic control using hand gestures [3]. Their work details a system that detects and classifies hand gestures in real time, enabling intuitive robotic manipulation. Emphasis is placed on the practical implementation of the recognition system, addressing issues like noise, variability in gesture execution, and ensuring that the system operates reliably in diverse environmental conditions.

Paterson and Aldabbagh (2021) present a method for controlling a robotic arm using gesture recognition implemented with OpenCV [4]. This paper describes a cost-effective solution that leverages conventional image processing techniques to detect gestures, which are then translated into commands for robotic movement. The approach highlights the potential for using readily available tools to build interactive systems that respond accurately to user gestures.

Peral et al. (2022) propose an efficient deep learning framework, known as EUREKA, for hand gesture recognition in human–robot interaction. Their system integrates hand landmark detection with temporal feature encoding and a multilayer perceptron for gesture classification. By optimizing the selection of key frames and network architecture, the authors achieve high accuracy and real-time performance, addressing the challenge of processing continuous gesture streams in dynamic settings.

Gao et al. (2021) introduce a dynamic hand gesture recognition method that leverages 3D hand pose estimation. Their approach fuses spatial and temporal information to better capture the dynamics of hand gestures, which is essential for applications in human–robot interaction [5]. The paper demonstrates that incorporating 3D pose data can significantly enhance the system's

ability to recognize complex gestures involving subtle movements and changes in hand configuration.

Zhang et al. (2017) explore the learning of spatiotemporal features for gesture recognition using a hybrid deep learning model [6]. Their work combines 3D convolutional neural networks (3DCNN) with convolutional long short-term memory (ConvLSTM) networks to simultaneously extract spatial features from individual frames and model temporal dependencies across sequences. This method shows promising results in recognizing both isolated and continuous gestures, making it a strong candidate for applications requiring robust temporal modeling.

Benitez-Garcia et al. (2021) contribute to the field by introducing the IPN Hand dataset, a benchmark designed for real-time continuous hand gesture recognition [7]. This dataset provides annotated video sequences of continuous gestures performed in realistic settings, enabling researchers to evaluate and compare the performance of different recognition systems under practical conditions. The availability of such a dataset is crucial for advancing research and developing systems that operate effectively in real-world scenarios.

Jo et al. (2023) enhance the interaction experience in virtual and augmented reality by developing a MediaPipe-based hand gesture recognition user interface [8]. Their system uses real-time hand tracking to facilitate intuitive control in immersive environments. The paper emphasizes the importance of low-latency and accurate gesture recognition to improve the naturalness and responsiveness of user interfaces in AR/VR applications.

Wan et al. (2016) discuss the ChaLearn “Looking at People” RGB-D datasets, which include both isolated and continuous gesture sequences [9]. This work underscores the significance of diverse datasets in training and evaluating gesture recognition systems. By providing a rich set of annotated examples captured under varying conditions, these datasets serve as a valuable resource for developing algorithms that can generalize well across different environments and user interactions.

Huu and Minh (2020) propose an artificial neural network-based gesture recognition algorithm tailored for smart-home applications [10]. Their system is designed to detect and classify dynamic hand gestures to control home appliances efficiently. The research addresses practical challenges such as varying illumination and background clutter, demonstrating that a well-designed ANN can achieve robust performance in real-life.

CHAPTER-3

PROPOSED SYSTEM

3.1 EFFICIENT HAND GESTURE RECOGNITION FOR HUMAN-ROBOT INTERACTION (2022)

In 2022, Peral and his team came up with a cool new way for robots to understand hand gestures. This method makes it easier for people to interact with robots without needing special devices like gloves or joysticks. Instead, it just uses a regular camera to see and recognize specific hand movements.

First, the system includes a Hand Landmark Extraction Module. This uses an algorithm called MediaPipe to find important points on the hand from video frames. MediaPipe works in two steps. First, it finds the hand using a special technique to draw a box around it. Then, it figures out where 21 key points are on the hand—like the tips of the fingers and the center of the palm. To make things faster, the system can remember hand positions from earlier frames, so it doesn't have to check everything again for each new frame.

Next, the Gesture Prediction Module. This part uses a special neural network to look at the hand landmarks and figure out what gestures are being made. This module is the heart of their system and can tell the difference between single gestures and smooth, continuous ones. It works at a speed of 10 frames per second and got an impressive accuracy of 87.5% on a test called the IPN Hand dataset. This is better than many other methods out there.

To recognize continuous gestures, Peral and his team created a batch-based processing approach. This means they take groups of gesture data (30, 60, or 90 frames) and process them all at once. Then, the system picks the prediction it is most confident about as the final answer. This helps the robots to understand flowing gestures without needing clear start or stop signals.

Finally, the system was tested with a social robot named IVO. The robot could see and react to human gestures in real time. It did a great job, even with changes in lighting or busy backgrounds. This strong performance shows that Peral's system could really work well in real-life situations where people interact with robots.

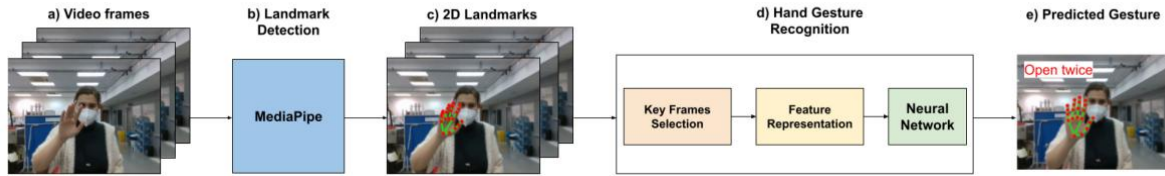


Fig. 3.1 EUREKA Architecture

3.1.1 DATASET

The IPN Hand dataset is a large collection of data used for recognizing hand gestures. It has 4,218 examples of hand movements and comes from 200 different videos. These videos show a variety of gestures performed by different people.

What's really interesting about this dataset is how diverse it is. The gestures are recorded in different settings, which include various lighting conditions and backgrounds. This makes it a little tricky for gesture recognition systems to learn. However, this challenge is what makes the IPN Hand dataset a great tool for testing models like EUREKA.

By using this dataset, EUREKA can learn to identify and understand gestures in everyday situations. As a result, it can work well in human-robot interactions, making them smoother and more accurate. This means EUREKA can respond effectively, even when the environment changes or the actions vary

3.1.2 WORKING OF EUREKA

● Key Frame Selection

Instead of processing every frame in a video sequence, EUREKA employs a key frame selection strategy known as DistTime (Distance over Time) to capture the most informative frames. The idea is to reduce redundancy and noise by selecting frames where significant changes occur in hand landmark positions. Mathematically, given a sequence of frames $f_1, f_2, \dots, f_L$ where each frame provides a set of N hand landmark coordinates, the system computes the Euclidean distance between corresponding landmarks in consecutive frames. For two consecutive frames, f_i and f_{i+1} , the distance for the k -th landmark is computed as:

$$d_{i,k} = \sqrt{(x_{i,k} - x_{i+1,k})^2 + (y_{i,k} - y_{i+1,k})^2}$$

A summary distance D_i for frame i can then be defined as:

$$D_i = \sum_{k=1}^N d_{i,k}$$

Frames with higher D_i values indicate greater motion, and by selecting frames at regular intervals where D_i exceeds a certain threshold, EUREKA effectively isolates key frames that encapsulate the critical dynamics of the gesture sequence.

● Feature Representation

After key frames are selected, EUREKA transforms the raw 2D hand landmark positions into a structured numerical feature representation. This transformation is essential to capture both the spatial configuration and the temporal evolution of the gesture. For each key frame, the raw input is a vector:

$$\mathbf{L} = [x_1, y_1, x_2, y_2, \dots, x_N, y_N]$$

where $N=21$ represents the number of landmarks. To incorporate temporal dynamics, the system may concatenate the feature vectors of M key frames, forming a composite vector:

$$\mathbf{F} = [\mathbf{L}_1, \mathbf{L}_2, \dots, \mathbf{L}_M]$$

This representation can be further normalized to achieve translation and scale invariance by subtracting the mean position or scaling by the hand's bounding box size. In some implementations, additional features, such as pairwise distances between landmarks, may be computed:

$$\text{Distance}_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

The final feature vector, encapsulating the spatial and temporal information of the gesture, is then used as the input for the deep neural network.

● **DNN-Based Classification**

The final stage of EUREKA employs a densely connected neural network, specifically a Multi-Layer Perceptron (MLP), for classifying gestures. The MLP architecture in EUREKA consists of an input layer, two hidden layers, and an output layer. Let $\mathbf{F}$ be the input feature vector derived from the selected key frames. The operation of each hidden layer can be expressed as:

$$\mathbf{h}^{(l)} = \text{ReLU} \left(\mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)} \right)$$

where $\mathbf{h}^{(0)} = \mathbf{F}$ is the input vector, $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are the weights and biases of layer l , and the ReLU function is defined as $\text{ReLU}(x) = \max(0, x)$. A dropout of 25% is applied during training to prevent overfitting, which involves randomly setting 25% of the activations to zero in each training iteration. The output layer uses a softmax activation to produce a probability distribution

over C gesture classes:

$$\hat{y}_j = \frac{e^{z_j}}{\sum_{k=1}^C e^{z_k}}, \quad \text{for } j = 1, 2, \dots, C$$

where z_j is the input to the softmax function for class j . The network is trained using a categorical cross-entropy loss function defined as:

$$\mathcal{L} = - \sum_{j=1}^C y_j \log(\hat{y}_j)$$

Here, y_j is the true label (one-hot encoded) for class j , and $\hat{y}_j$ is the predicted probability. The optimization is performed using stochastic gradient descent (SGD) with a learning rate scheduler to ensure stable convergence.

By combining these three stages—key frame selection to isolate significant temporal changes, feature representation to encode spatial and temporal dynamics, and DNN-based classification to map features to gesture categories—EUREKA achieves robust and real-time gesture recognition. This modular approach enables the system to operate at 10 frames per second and attain an accuracy of 87.5% on the IPN Hand dataset, demonstrating its effectiveness in continuous hand gesture recognition for human-robot interaction.

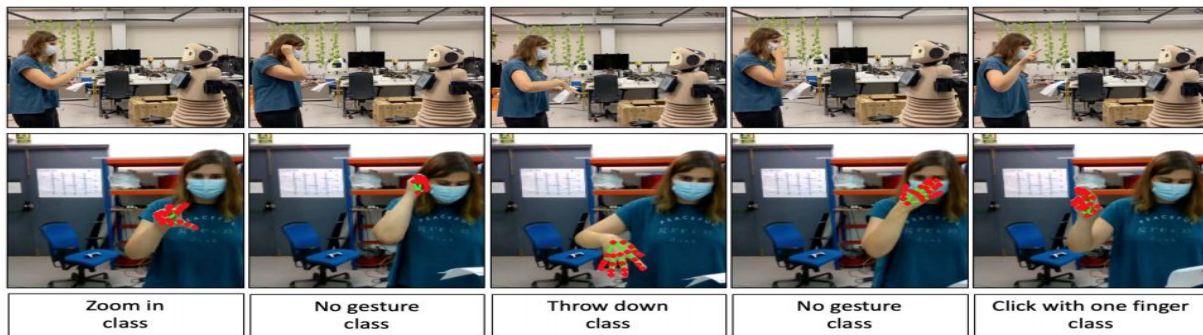


Fig. 3.2 hand-gestures performed in front of IVO

3.2 HAND GESTURES ROBOTIC CONTROL BASED ON COMPUTER VISION (2023)

Wameed and his team (2023) created a cool robot that you can control with your hands! They used a special method called the MediaPipe algorithm along with machine learning to recognize hand gestures in real time. This means you can move the robot just by waving your hands.

Their system combines different technologies like computer vision and machine learning. They also use hardware, which is the physical part of the device. These elements work together to make a fun and responsive way to control a robot.

One of the key features of this setup is the ability to detect and understand hand gestures quickly. The team designed it to adapt to the environment, which helps the robot respond better to changes around it.

To make it all work, they used a Raspberry Pi and an Arduino. These tools help the robot process gestures quickly and react without delay. This makes the system useful for both home use and in factories.

Overall, this gesture-based robot control system shows how fun and practical technology can be. It makes controlling robots easier and more exciting for everyone!

3.2.1 HAND GESTURE DETECTION AND CLASSIFICATION

In the hand gesture detection and classification phase, the system relies on the **MediaPipe algorithm** to detect and track hand movements in real time. The MediaPipe framework follows a three-step process:

- **Palm Detection**

MediaPipe uses a special method called single-shot detection, or SSD, to find palms in pictures. When you upload a frame, the palm detection model creates a box around the palm. This box helps the system figure out where the hand is and find specific points, known as landmarks, on the palm. What makes MediaPipe's method different from older hand detection techniques is its ability to handle situations where hands are partly hidden or blocked. This means it works better in real-life situations where hands might not always be fully visible.

- **Hand Landmark Estimation**

After MediaPipe detects your palm, it figures out the 3D positions of 21 points on your hand. These points include important spots like your fingertips, knuckles, the center of your palm, and the base of your thumb. The system calculates where each landmark is in three dimensions, using X, Y, and Z coordinates. The Z-coordinate is especially useful because it tells us how far away your hand is, helping track your movements in 3D space.

To find the position of these landmarks, MediaPipe uses a method that learns from the features in the input image. This means that it can estimate where each point is on your hand based on what it sees in the picture. Moreover, the hand model is trained to work with all kinds of hands. It can adapt to different sizes, shapes, and angles. This makes it easier for various people to use the system, no matter how unique their hands may be.

- **Gesture Classification**

The extracted hand landmarks are processed by a machine learning model, which classifies the gestures based on the relative positions and movements of the landmarks. The system maps each recognized gesture to a specific robot control command.

The Form of Hand Gestures				
1	2	3	4	5
				
The Commands Carried Out in the Robot				
Stop (Robot is not moving)	Move forward	Move backward	Left (Rotating anti clockwise)	Right (Rotating clockwise)

Fig. 3.3 Gestures

3.2.2 ROBOT CONTROL

Once the system recognizes a gesture, it translates that gesture into a command for the robot. This command tells the robot what to do. The Raspberry Pi 4 sends the command wirelessly to an Arduino Mega using a Bluetooth module called HC-05. This means they can talk to each other without any wires getting in the way. When the Arduino Mega gets the command, it controls the robot's movements. It does this using two DC motors.

These motors need a lot of power to work, so the Arduino uses a motor driver called BTS7960. This driver takes the small signals from the Arduino and turns them into strong signals that can drive the motors. In this way, all the parts work together to help the robot move based on the gestures it recognizes. It's all about teamwork between the Raspberry Pi, Arduino, and the motors! The robot control mechanism includes the following components:

- Motor Driver (BTS7960)
- DC Motors
- HC-SR04 Ultrasonic Sensor

The robot executes movement commands immediately after gesture recognition, ensuring low latency and high responsiveness. The use of wireless communication enables flexible and untethered robot operation.

3.2.3 HARDWARE AND SOFTWARE COMPONENTS

The system's hardware and software components are designed to ensure real-time performance and low-cost implementation:

Hardware:

- Raspberry Pi 4 (8 GB)
- Arduino Mega
- HC-05 Bluetooth Module
- BTS7960 Motor Driver
- HC-SR04 Ultrasonic Sensor
- DC Motors

Software:

- Python
- OpenCV
- MediaPipe

■ Arduino c

The system's software integrates real-time video processing with gesture classification and robot control, ensuring seamless operation.

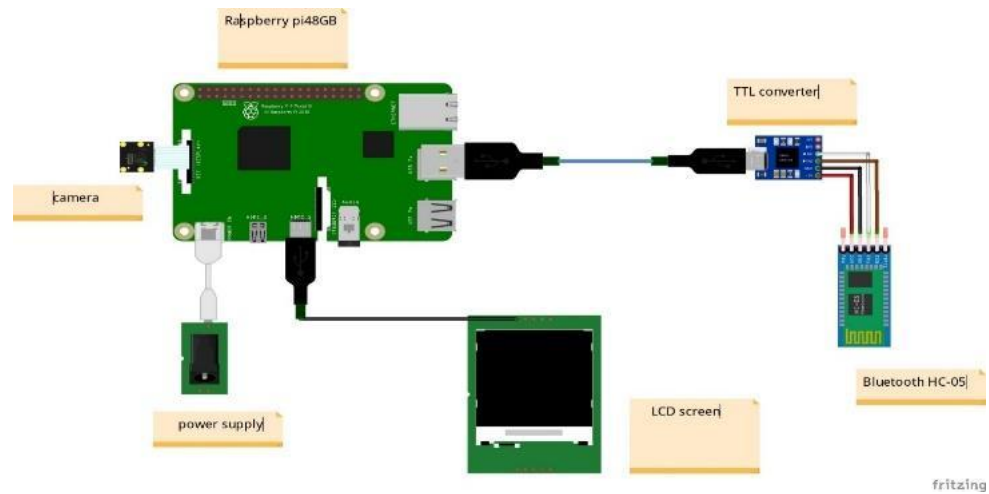


Fig. 3.4 Base station

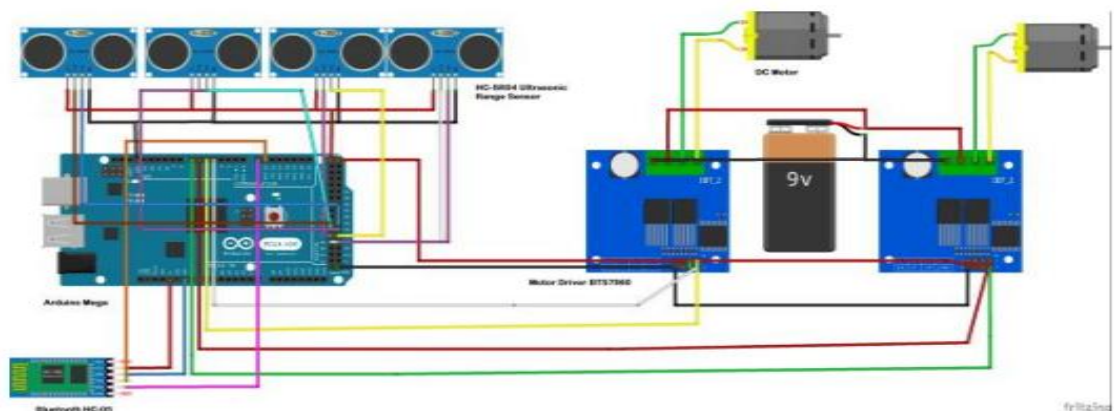


Fig. 3.5 The assembled Tank robot

CHAPTER-4

METHODOLOGY

4.1 MediaPipe

MediaPipe is a free tool created by Google. It helps computers see and understand images and videos in real time. You can use it for tasks like finding faces, tracking hands, and recognizing actions. It's popular because it works quickly, doesn't need a lot of computer power, and can run on different types of devices.

MediaPipe uses a special process called a pipeline. Imagine it as a series of steps that data goes through. For hand tracking, MediaPipe has two important parts: the Palm Detector and the Hand Landmark Model. The Palm Detector finds the hand in a picture and outlines it with a box. Then, the Hand Landmark Model spots 21 key points on the hand, like where the fingers and palm are. This model uses smart technology to predict where these points should be based on the image.

Even though the full details of MediaPipe's training process aren't completely available, it's obvious that it makes use of specialized neural networks to accurately identify hand landmarks. By training on a wide variety of hand images, it ensures consistent performance across different hand shapes and lighting conditions. For example, in study by Peral et al. (2022), researchers used MediaPipe to find 2D hand locations from videos. First, the Palm Detector found the hand, and then the Hand Landmark Model identified the 21 important points on the hand. MediaPipe's strength was its ability to track fast hand movements at 10 frames per second, even with changing light and background.

Similarly, Wameed et al. (2023) used MediaPipe to monitor hand movements in real time. The information about the 21 hand points was sent to a machine learning system that figured out the gestures. The tool was effective at handling various lighting, distances, and hand positions. This made it reliable for controlling robots using gestures.

In summary, MediaPipe is a powerful tool that helps computers understand hand movements quickly and accurately. Its ability to adapt to different conditions makes it useful in many projects, especially where real-time responses are essential.

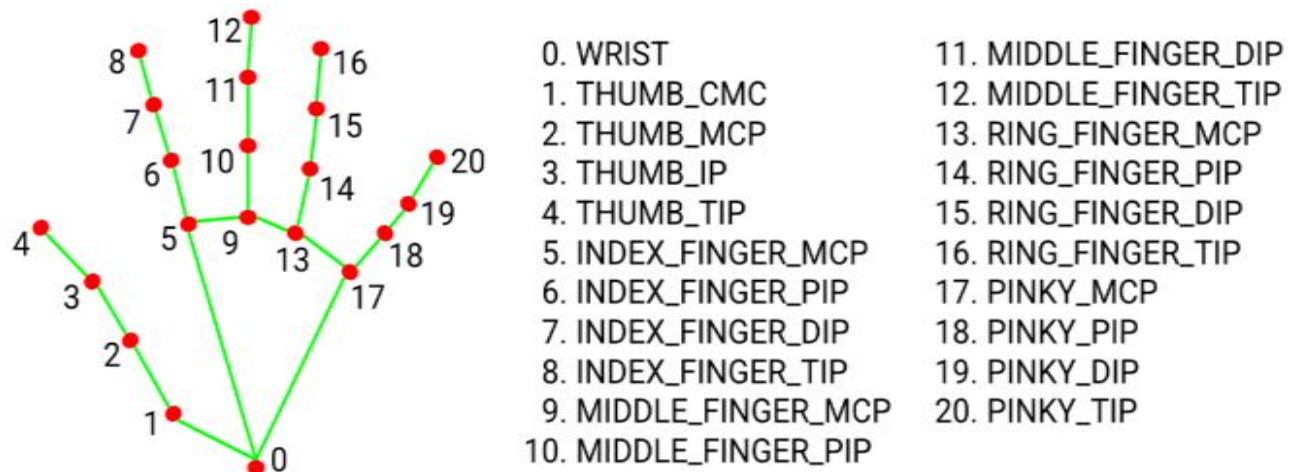


Fig. 4.1 20 hand landmarks

4.2 OpenCV

OpenCV stands for Open Source Computer Vision. It's a free library that helps computers see and understand images. People use it for real-time image processing, which means looking at pictures or videos instantly. OpenCV is also great for object detection, which is tagging different things in images, and pattern recognition, where the computer finds and matches patterns in what it sees.

The library offers a lot of helpful tools for analyzing images. For example, it can find edges in pictures, which helps define shapes. It can also filter colors, match templates, and extract important features from images. These functions are useful in many tech applications, from making apps to developing robots.

Even though OpenCV is mainly written in C++, it has support for other programming languages too, like Python and Java. This means many people can use it, no matter what platform they prefer. Whether you're working on a school project or a professional app, OpenCV can be a powerful tool.

OpenCV itself is not a trainable framework like MediaPipe, but it provides tools for preprocessing and analyzing image data used in machine learning models. In the context of gesture recognition, OpenCV is used to:

- Capture video frames from a camera.
- Convert color spaces (e.g., from RGB to grayscale).

- Apply noise reduction and smoothing.
- Detect edges and contours.
- Apply transformations like rotation, cropping, and scaling.

In the study by Peral et al. (2022), researchers used a tool called OpenCV to help prepare video frames for another program called MediaPipe. This tool made the videos clearer by resizing the images, adjusting how bright or dark they appeared, and removing any extra noise. OpenCV also helped capture video from the camera in real time, which made sure the input quality was good for recognizing hand gestures.

Similarly, Wameed et al. (2023) also relied on OpenCV for working with video. They used it to capture video and process images as well. This meant resizing the input frames and filtering them to enhance clarity before sending them to the MediaPipe model to find hand landmarks.

Moreover, OpenCV was crucial in keeping everything running smoothly. It ensured that the gesture recognition worked well, even when the surroundings changed. This means users could make hand gestures naturally, without worrying about technology not keeping up.

4.3 PYTHON

Python is a popular programming language that is easy to read and understand. It is known for being simple, which makes it a great choice for beginners and experienced programmers alike. This language can be used in many different ways. It supports various styles of programming, such as object-oriented, functional, and procedural programming. Because of this, Python is very flexible and can be used for many types of projects.

Many people use Python in exciting fields like data science, artificial intelligence, and machine learning. This is largely because it has a lot of helpful libraries and tools. Some of these are called TensorFlow, PyTorch, OpenCV, and MediaPipe, which are great for building advanced programs. One of the best things about Python is how it manages memory and updates itself while you code. This feature helps developers work quickly and make programs faster. With Python, you can easily create and test complex ideas without too much hassle.

Peral et al. (2022) used Python to implement the EUREKA framework for hand gesture recognition. Python was used for several key tasks:

- **MediaPipe Integration:** Python was used to load and process hand landmark data using the MediaPipe library.
- **OpenCV for Image Processing:** OpenCV (in Python) was used to handle video frame capture, resizing, and preprocessing.
- **Machine Learning Framework:** Python-based libraries such as TensorFlow and PyTorch were used to build and train the densely connected neural network used in the Gesture Prediction Module.
- **Keyframe Selection Strategy:** Python was used to implement the DistTime algorithm to select keyframes based on the movement of hand landmarks over time.
- **Performance Optimization:** Python's ability to handle large datasets and support for matrix operations allowed the model to process gesture data at 10 frames per second.

Wameed et al. (2023) also used Python as the main programming language for developing the gesture-based robot control system. Python's role in their system included:

- **MediaPipe Integration:** Python was used to implement the hand landmark extraction pipeline using MediaPipe.
- **OpenCV for Video Processing:** OpenCV (in Python) was used to handle real-time video capture and preprocessing.
- **Machine Learning:** Python-based libraries were used to implement and train the gesture classification model.
- **Control Logic:** Python was used to process classified gestures and translate them into robot control commands.
- **Bluetooth Communication:** Python was used to implement the Bluetooth communication protocol between the Raspberry Pi and Arduino for transmitting gesture commands.
- **Performance Optimization:** Python's real-time processing capabilities enabled the system to maintain high responsiveness even under varying environmental conditions.

4.4 ARDUINO

Arduino is a cool electronics platform that anyone can use. It combines simple hardware and software, making it easy for beginners to get started with electronics projects. At its core, Arduino has a small board called a microcontroller. This is a tiny processor that can work with different types of signals, both digital and analog. In simpler terms, it means the board can read inputs like sensors and control outputs like lights and motors.

To make things even easier, Arduino comes with a special program called the Integrated Development Environment, or IDE. This is where you can write and upload your code to the board. With just a few lines of code, you can make your projects come to life. Arduino boards can also talk to other devices through different communication methods like I2C, SPI, and UART. This means you can connect sensors and other parts easily, which helps with real-time control of things like motors.

In Wameed et al. (2023) system Arduino played a crucial role in Wameed et al.'s robot control system.

- **Motor Control:** An Arduino Mega was used to receive gesture commands from the Raspberry Pi and translate them into motor control signals.
- **Motor Driver (BTS7960):** The Arduino controlled the motor driver to regulate the speed and direction of the robot's DC motors.
- **Ultrasonic Sensor:** The Arduino handled data from the HC-SR04 ultrasonic sensor to enable obstacle detection and avoidance.
- **Bluetooth Communication:** The Arduino received gesture commands from the Raspberry Pi through the HC-05 Bluetooth module and converted them into real-time robot movements.
- **Real-Time Response:** The Arduino's low-level control allowed quick response to incoming signals, ensuring smooth and accurate robot operation.

CHAPTER 5

EXPERIMENTS & RESULTS

5.1 EFFICIENT HAND GESTURE RECOGNITION FOR HUMAN-ROBOT INTERACTION (2022)

Peral et al. (2022) conducted a series of experiments to see how well EUREKA performed, the team used a special dataset called the IPN Hand dataset. This dataset includes 4,218 examples of different hand gestures taken from 200 videos. These videos show a wide variety of hand movements, making the tests very thorough.

Overall, the researchers wanted to find out if the EUREKA framework could quickly and accurately recognize hand gestures in different situations. Their work could help improve technology that uses hand gestures, making it more reliable and effective for users.

Dataset and Training

- The model was trained on the IPN Hand dataset using supervised learning.
- The input data consisted of 21 hand landmarks extracted using MediaPipe, converted into a temporal feature representation.
- The network was optimized using Stochastic Gradient Descent (SGD) with a learning rate scheduler.

Accuracy

- The system achieved an overall accuracy of 87.5% on the IPN Hand dataset, outperforming existing state-of-the-art models.
- The use of the keyframe selection strategy (DistTime) significantly improved the system's ability to filter noise and focus on meaningful motion patterns, thereby enhancing classification accuracy.

Model	Accuracy
C3D	0.778
ResNeXt-101	0.836
ResNet-50	0.731
DistTime	0.875
DistAndTime	0.869

Table. 5.1 ACCURACY WITH IPN HAND DATASET

Continuous Gesture Handling

- The batch-based processing approach allowed the system to process gestures of varying lengths without requiring explicit start and end signals.
- The system was tested with three different batch sizes: **30, 60, and 90 frames**.
- The prediction with the highest confidence score across batches was selected as the final gesture output.

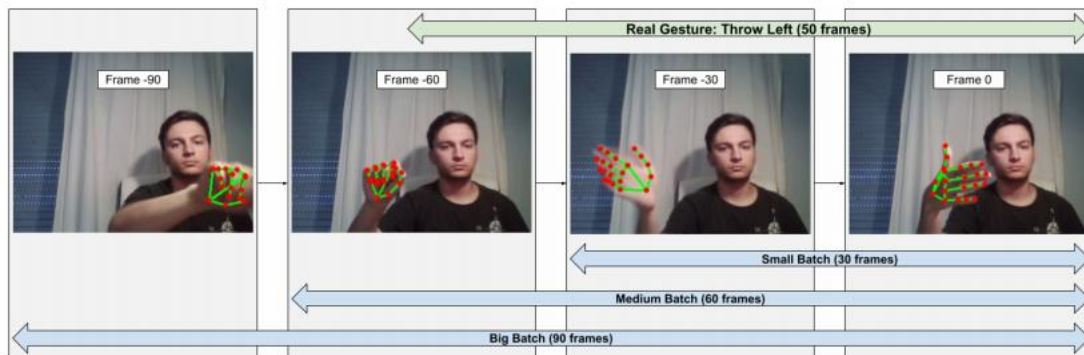


Fig. 5.1 Batches with 30, 60 and 90 frames are taken

Processing Speed

- The system operated at **10 frames per second**, ensuring real-time gesture recognition.
- The efficient use of keyframe selection and batch processing enabled the system to handle complex gestures without compromising response time.

Testing with IVO Robot

- The system was integrated with the IVO robot to test its real-world performance.
- The robot successfully recognized and responded to human gestures in real time.
- High accuracy was maintained even under varying lighting conditions and complex background movements.

5.2 HAND GESTURES ROBOTIC CONTROL BASED ON COMPUTER VISION (2023)

Wameed et al. (2023) ran thorough tests to judge the correctness, reaction speed along with environmental flexibility of their robotic control method, which used gestures. The method went through tests in differing light levels, separations as well as hand placements. This was to act like conditions in the actual world. The tests were centered on the rightness of recognizing the gestures along with how well the robot did at control.

Dataset and Training

- The model was trained using hand landmark data extracted by MediaPipe.
- The input consisted of 21 hand landmarks (in X, Y, and Z coordinates).
- The machine learning model was trained using supervised learning, with the input landmarks mapped to specific robot commands (forward, backward, left, right, and stop).

Accuracy

- The system achieved **100% accuracy** under ideal conditions (100 lux lighting, 1m distance, and 0° tilt).
- Performance slightly declined under low lighting, long distances, and extreme tilt angles.

Environmental Adaptation

1. Lighting Conditions

- 50 lux – Moderate performance (slight reduction in accuracy).
- 100 lux – Maximum accuracy (100% recognition rate).

- 150 lux – Slight reduction due to glare and overexposure.

Hand Gesture	Accuracy with Intensity Light (%)		
	50 LUX	100 LUX	150 LUX
Stop	100	100	100
Forward	100	100	100
Backward	100	100	100
Left	75	100	65
Right	65	100	40

Table. 5.2 Light intensity variation based accuracy

2. Distance from Camera

- 1m – Maximum accuracy due to clear landmark visibility.
- 2m – Slight reduction in accuracy due to decreased resolution.
- 4m – Noticeable drop in accuracy due to poor landmark tracking.

Hand Gesture	Accuracy with Distance (%)		
	1 m	2 m	4m
Stop	100	100	100
Forward	100	100	100
Backward	100	100	100
Left	100	80	70
Right	100	70	70

Table. 5.3 Variations in distance-based accuracy

3. Tilt Angles

- 0° – Maximum accuracy.
- 30° – High accuracy maintained.
- 60° – Reduced accuracy due to hand distortion.
- 90° – Noticeable drop in accuracy due to hand occlusion.

Hand Gesture	Accuracy with Distance (%)			
	0°	30°	60°	90°
Stop	100	100	100	60
Forward	100	100	100	30
Backward	100	100	100	20
Left	100	100	55	15
Right	100	100	40	12

Table. 5.4 Tilt-angle testing accuracy

Response Time

- The system maintained a response time of <200 ms between gesture recognition and robot action.
- The Bluetooth-based wireless communication between the Raspberry Pi and Arduino ensured quick signal transmission.

Obstacle Avoidance

- The ultrasonic sensor (HC-SR04) was tested for obstacle detection and avoidance.
- The robot successfully stopped or changed its path when obstacles were detected.

Hand Gesture	Average Reaction Time (s)
Stop	0.091
Forward	0.121
Backward	0.126
Left	0.162
Right	0.165
Average Total Reaction Time (s)	0.133

Table. 5.5 The Robotic Response time test

Real-World Performance

- The system was tested in dynamic environments with different backgrounds and lighting variations.
- The robot consistently executed the correct commands based on recognized gestures.
- The system maintained high responsiveness even when multiple users were present in the background.

CHAPTER 6

COMPARISON RESULTS

Metric	PAPER 1(2022)	PAPER 2(2023)
Accuracy	87.5%	100%
Processing Speed	10 frames per second	<200 ms response time
Lighting Conditions	Robust under varying conditions	Best at 100 lux; reduced at low/high light
Distance	Not specified	Optimal at 1m; drop at 4m
Tilt Angles	Handled well with keyframe selection	Accurate up to 60°; drop at 90°
Continuous Gesture Handling	Batch-based approach with 30, 60, 90 frames	Not designed for continuous gestures
Real-World Testing	Successful with IVO robot	Successful with tracked robot
Challenges	Reduced performance under extreme lighting and occlusion	Drop at high tilt angles and low lighting

Table. 6.1 Comparison table

CHAPTER 7

CONCLUSION

Gesture-based human-robot interaction (HRI) is becoming an exciting way for people and robots to communicate naturally. This report looks at two advanced gesture recognition systems made by Peral et al. in 2022 and Wameed et al. in 2023c. Both systems use computer vision and machine learning to recognize gestures quickly and accurately.

First, let's talk about Peral et al.'s system. They created an efficient deep-learning framework for recognizing gestures that uses a tool called the EUREKA framework. This system picks out hand movements by combining data from MediaPipe with a special type of neural network. Their approach, called DistTime, helps the system track hand movements over time. It achieved an accuracy rate of 87.5% on the IPN Hand dataset. The system worked well for both simple and complex gestures in real-time, even when tested on the IVO robot in different lighting conditions. This shows it can adapt to various human-robot interaction situations. On the other hand, Wameed et al. focused on a real-time robotic control system for gestures. They also used MediaPipe to capture hand movements and built a machine learning model to identify gestures. Their system runs on a Raspberry Pi 4 and Arduino Mega, allowing for efficient robot control. In perfect conditions, it reached a perfect score of 100% accuracy. Even with changes in lighting, distance, and angles, it performed well. They added an ultrasonic sensor to help the robot avoid obstacles and used Bluetooth for quick responses to commands based on gestures.

Both Peral et al. and Wameed et al. showed how gesture-based systems can improve human-robot interactions. While Peral et al. focused on recognizing continuous gestures better, Wameed et al. worked on making robots respond in real-world situations. However, some challenges still need to be addressed, such as lighting changes and obstacles blocking hand movements. Future advancements could include techniques that adapt to changing conditions and improve the system's overall performance.

In summary, combining computer vision, machine learning, and hardware opens up exciting possibilities for gesture-based human-robot interaction. The findings from these two studies stress the importance of recognizing gestures accurately and processing them quickly. This can lead to robots that behave more naturally and interact more intelligently with humans.

CHAPTER-8

REFERENCES

- [1] Peral, Marc, Alberto Sanfeliu, and Anaís Garrell. "Efficient hand gesture recognition for human-robot interaction." *IEEE Robotics and Automation Letters* 7.4 (2022): 10272-10279.
- [2] Qi, Jing, et al. "Computer vision-based hand gesture recognition for human-robot interaction: a review." *Complex & Intelligent Systems* 10.1 (2024): 1581-1606.
- [3] Wameed, M., and A. M. Alkamachi. "Hand gestures robotic control based on computer vision." *International Journal of Intelligent Systems and Applications in Engineering* 11.2 (2023): 1013-1021.
- [4] Paterson, Jedidiah, and Ahmed Aldabbagh. "Gesture-controlled robotic arm utilizing opencv." *2021 3rd International Congress on Human-Computer Interaction, Optimization and Robotic Applications (HORA)*. IEEE, 2021.
- [5] Gao, Qing, et al. "Dynamic hand gesture recognition based on 3D hand pose estimation for human-robot interaction." *IEEE Sensors Journal* 22.18 (2021): 17421-17430.
- [6] Zhang, Liang, et al. "Learning spatiotemporal features using 3dcnn and convolutional lstm for gesture recognition." *Proceedings of the IEEE international conference on computer vision workshops*. 2017.
- [7] Benitez-Garcia, Gibran, et al. "Ipn hand: A video dataset and benchmark for real-time continuous hand gesture recognition." *2020 25th international conference on pattern recognition (ICPR)*. IEEE, 2021.
- [8] Jo, Beom Jun, Seok-Kyoo Kim, and S. Kim. "Enhancing virtual and augmented reality interactions with a mediapipe-based hand gesture recognition user interface." *Ing. Syst. D'Inf* 28 (2023): 633-638.
- [9] Wan, Jun, et al. "Chalearn looking at people rgb-d isolated and continuous datasets for gesture recognition." *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*. 2016.
- [10] Huu, Phat Nguyen, and Quang Tran Minh. "An ANN-based gesture recognition algorithm for smart-home applications." *KSII Transactions on Internet and Information Systems (TIIS)* 14.5 (2020): 1967-1983.